



YZM212 – Makine Öğrenmesinde Özdeğer ve Özvektör Kavramları

İsim: Furkan Büyükyozgat

Öğrenci No: 23291050

Tarih: 29/03/2026

Makine Öğrenmesinde Matris Manipülasyonu, Özdeğerler ve Özvektörler

Lineer cebir, makine öğrenmesinin matematiksel temelini oluşturan başlıca alanlardan biridir. Özellikle matrisler, veri kümelerinin gösterimi, doğrusal dönüşümlerin modellenmesi, optimizasyon işlemleri ve boyut indirgeme tekniklerinde merkezi bir rol üstlenmektedir. Özdeğer ve özvektör kavramları ise bir doğrusal dönüşümün uzay üzerindeki etkisini anlamada temel araçlardır. Bir kare matris için $(A v = \lambda v)$ eşitliğini sağlayan sıfırdan farklı (v) vektörü özvektör, buna karşılık gelen (λ) skaler değeri ise özdeğer olarak adlandırılır.

Bu çalışmada ilk olarak matris manipülasyonu, özdeğerler ve özvektörlerin makine öğrenmesiyle ilişkisi açıklanmıştır. Ardından NumPy kütüphanesindeki `numpy.linalg.eig` fonksiyonu, dokümantasyonu ve kaynak kodu çerçevesinde incelenmiştir. Son bölümde ise verilen GitHub deposunda yer alan manuel özdeğer hesaplama yaklaşımı yeniden uygulanmış ve NumPy'nin hazır fonksiyonuyla elde edilen sonuçlarla karşılaştırılmıştır.

1. Makine Öğrenmesi ile Matris Manipülasyonu, Özdeğerler ve Özvektörlerin İlişkisi

1.1. Temel kavramlar

Matris, sayısal verilerin satır ve sütunlar biçiminde düzenlenmiş gösterimidir. Makine öğrenmesinde veri kümeleri çoğu zaman matris biçiminde temsil edilir; satırlar gözlemleri, sütunlar ise özellikleri ifade eder. Bu nedenle matris toplama, çıkarma, çarpma, transpoz alma, ters matris bulma ve benzeri işlemler hem teorik hem de uygulamalı açıdan büyük önem taşır. Bu işlemler genel olarak matris manipülasyonu kapsamında değerlendirilir.

Özdeğer ve özvektör kavramları, bir matrisin belirli doğrultularda nasıl davrandığını incelemek için kullanılır. Bir doğrusal dönüşüm, bazı vektörlerin yönünü değiştirmeden yalnızca büyüklüğünü ölçeklendiriyorsa, bu vektörler özvektör; bu ölçekleme katsayıları ise özdeğer olarak tanımlanır. Bu bağlamda özdeğerler ve özvektörler, bir matrisin iç yapısının anlaşılmasına imkân tanır.

1.2. Makine öğrenmesindeki önemi

Makine öğrenmesi uygulamalarında veriler, model parametreleri ve dönüşümler çoğunlukla matrisler aracılığıyla ifade edilir. Özellikle doğrusal regresyon, lojistik regresyon, temel bileşen analizi, boyut indirgeme yöntemleri ve sinir ağları gibi birçok yaklaşım matris işlemlerine dayanır. Büyük veri kümeleri üzerinde toplu işlem yapılabilmesi, matris temelli gösterimin sağladığı temel avantajlardan biridir.

Özdeğer ve özvektörler ise özellikle veri içindeki baskın yönlerin ve yapısal özelliklerin belirlenmesinde kullanılır. Bunun en bilinen örneklerinden biri Temel Bileşen Analizi'dir (Principal Component Analysis, PCA). PCA yönteminde kovaryans matrisi üzerinde yapılan özdeğer-özvektör analizi ile verideki en yüksek varyansa sahip doğrultular belirlenir. En büyük özdeğerlere karşılık gelen özvektörler, verinin en fazla bilgi taşıyan bileşenlerini temsil eder. Böylece veri, bilgi kaybı en aza indirilerek daha düşük boyutlu bir uzayda ifade edilebilir.

Bunun yanı sıra spektral kümeleme, grafik tabanlı öğrenme yöntemleri, doğrusal dinamik sistemlerin analizi ve bazı optimizasyon problemleri de özdeğer-özvektör yaklaşımından yararlanmaktadır. Bu nedenle matris manipülasyonu ile özdeğer-özvektör analizi, makine öğrenmesinde hem kuramsal hem de hesaplamalı açıdan birbirini tamamlayan iki temel unsurdur.

1.3. Kullanım alanları ve yaklaşımlar

Makine öğrenmesinde matris manipülasyonu, özdeğerler ve özvektörlerin kullanıldığı başlıca alanlar aşağıda özetlenebilir:

Temel Bileşen Analizi (PCA): Boyut indirgeme, gürültü azaltma ve veri görselleştirme amacıyla kullanılır.

Spektral yöntemler: Özellikle benzerlik matrisleri veya Laplace matrisleri üzerinden çalışan kümeleme yöntemlerinde etkilidir.

Doğrusal dönüşüm analizi: Bir matrisin uzay üzerindeki etkisini anlamak, kararlılık incelemesi yapmak ve sistem davranışını çözümlemek için kullanılır.

Sayısal lineer cebir uygulamaları: Büyük ölçekli problemlerde özdeğer-özvektör hesabı doğrudan teorik çözüm yerine sayısal algoritmalar ile gerçekleştirilir. NumPy gibi kütüphaneler bu noktada yüksek verimlilik sağlar.

2. NumPy linalg.eig Fonksiyonunun İncelenmesi

2.1. Fonksiyonun amacı

`numpy.linalg.eig(a)` fonksiyonu, kare bir matrisin özdeğerlerini ve sağ özvektörlerini hesaplamak amacıyla kullanılmaktadır. Fonksiyonun çıktısı iki bileşenden oluşur: ilki özdeğerleri içeren bir dizi, ikincisi ise sütunları ilgili özvektörleri temsil eden bir matristir. Başka bir ifadeyle, çıktındaki her sütun vektörü, aynı sıradaki özdeğere karşılık gelir.

Fonksiyon yalnızca kare matrisler için tanımlıdır. Yakınsama sağlanamadığında lineer cebir hatası üretmektedir. Ayrıca genel kare matrisler için `eig`, simetrik ya da Hermitian yapılar için ise `eigh` gibi daha özel fonksiyonların tercih edilmesi önerilmektedir.

2.2. Dokümantasyon temelli değerlendirme

NumPy dokümantasyonunda `eig` fonksiyonunun temel matematiksel bağıntısı şu biçimde ifade edilmektedir:

$$Av_i = \lambda_i v_i$$

Burada (λ_i) özdeğeri, (v_i) ise karşılık gelen sağ özvektörü göstermektedir. Dokümantasyona göre döndürülen özvektörler normalize edilmiş durumdadır. Ayrıca özdeğerlerin belirli bir sırayla döndürülmesinin garanti edilmediği belirtilmektedir. Bu durum, aynı matris için farklı yöntemler kullanıldığında özdeğerlerin sıralamasında farklılık gözlenebileceği anlamına gelir.

Gerçek katsayılı matrislerde bazı durumlarda karmaşık özdeğerler ve özvektörler ortaya çıkabilmektedir. NumPy, bu tür durumlarda sonuçları uygun veri tipi ile döndürmektedir.

Eğer karmaşık kısmın sıfır olduğu anlaşılırsa sonuç uygun biçimde gerçek tipe indirgenebilmektedir.

2.3. Kaynak kod düzeyinde inceleme

NumPy'nin lineer cebir modülünde eig ve eigvals fonksiyonları, yüksek seviyeli Python arayüzü ile LAPACK tabanlı düşük seviyeli sayısal yordamlar arasında bir köprü görevi görmektedir. Kaynak kod incelendiğinde, öncelikle giriş matrisinin uygunluğu denetlenmektedir. Bu kapsamda matrisin en az iki boyutlu olması, kare yapıda bulunması ve sonlu sayısal değerler içermesi şartları kontrol edilmektedir.

Ardından giriş verisinin tipi belirlenmekte ve buna uygun hesaplama yolu seçilmektedir. Özdeğer ve özvektör hesabı için arka planda `_umath_linalg.eig` benzeri alt seviye yordamlar çağrılmaktadır. NumPy dokümantasyonu, bu işlemin LAPACK'ın `_geev` rutinlerine dayandığını belirtmektedir. Böylece doğrudan karakteristik polinomun sembolik çözümüne gidilmeden, nümerik olarak daha verimli ve kararlı bir çözüm elde edilmektedir.

Fonksiyonun işleyiş mantığı şu şekilde özetlenebilir:

1. Girdi dizisinin uygunluğu denetlenir.
2. Veri tipine göre gerçek veya karmaşık hesaplama yolu seçilir.
3. Alt seviye lineer cebir yordamı çağrılarak özdeğerler ve özvektörler hesaplanır.
4. Gerekli durumda sonuç tipi sadeleştirilir.
5. Özdeğerler ve özvektörler birlikte döndürülür.

Bu yapı, hem kullanıcı dostu bir arayüz sunmakta hem de yüksek performanslı sayısal kütüphanelerin avantajlarını korumaktadır.

2.4. Sayısal yöntem açısından değerlendirme

`numpy.linalg.eig` fonksiyonu, küçük boyutlu örneklerin ötesinde pratik uygulamalarda tercih edilmesi gereken standart araçlardan biridir. Bunun temel nedenleri; hesaplama hızının yüksek olması, büyük matrislerde daha ölçeklenebilir davranması ve sayısal kararlılık bakımından güçlü bir altyapıya dayanmasıdır. Özellikle teorik olarak determinant temelli yöntemler öğretici olmakla birlikte, matris boyutu arttıkça bu yöntemlerin maliyeti hızla yükselmektedir. NumPy'nin LAPACK tabanlı yaklaşımı ise uygulama düzeyinde daha uygundur.

3. Manuel Özdeğer Hesabı ve NumPy ile Karşılaştırılması

3.1. Repository'deki yaklaşımın genel yapısı

Verilen GitHub deposunda yer alan yaklaşım, özdeğer hesabını NumPy'nin hazır eig fonksiyonunu doğrudan kullanmadan gerçekleştirmektedir. Temel mantık, karakteristik matrisin oluşturulması, determinantın polinom biçiminde elde edilmesi ve ardından bu polinomun köklerinin bulunmasına dayanmaktadır. Bu yönüyle yöntem, özdeğer

kavramının doğrudan karakteristik denklem üzerinden nasıl ortaya çıktığını göstermektedir.

Söz konusu yaklaşımın başlıca adımları şunlardır:

1. Matris boyutunun belirlenmesi
2. Birim matrisin oluşturulması
3. $(A - \lambda I)$ biçimindeki karakteristik yapının kurulması
4. Determinantın rekürsif biçimde polinom olarak hesaplanması
5. Elde edilen karakteristik polinomun köklerinin bulunması

Bu yöntem, kuramsal anlamda öğretici ve açıklayıcıdır. Ancak determinant hesabının rekürsif biçimde yürütülmesi, matris boyutu büyüdükçe hesaplama maliyetini artırmaktadır.

3.2. Uygulamada kullanılan matris

Karşılaştırma çalışmasında aşağıdaki matris kullanılmıştır:

$$A = \begin{bmatrix} 6 & 1 & -1 \\ 0 & 7 & 0 \\ 3 & -1 & 2 \end{bmatrix}$$

Bu matris üzerinde hem NumPy'nin hazır yöntemi hem de karakteristik polinom temelli manuel yöntem uygulanmıştır.

3.3. Manuel yöntemin işleyişi

Manuel uygulamada önce matrisin boyutu elde edilmekte, ardından aynı boyutta birim matris oluşturulmaktadır. Daha sonra karakteristik denklem için $(A - \lambda I)$ yapısı kurulmaktadır. Determinant hesabı doğrudan sayısal değil, polinom katsayıları üzerinden yürütülmektedir. Bu nedenle polinom toplama ve çarpma işlemleri için ayrı yardımcı fonksiyonlara ihtiyaç duyulmaktadır.

2x2 durumunda determinant klasik formül ile hesaplanırken, daha büyük boyutlu matrislerde bir satır veya sütun üzerinden kofaktör açılımı yapılarak rekürsif biçimde determinant polinomu elde edilmektedir. Son aşamada karakteristik polinomun katsayıları kullanılarak kökler bulunmakta ve bu kökler özdeğerler olarak yorumlanmaktadır.

Bu yaklaşım, özdeğer hesabının ardındaki matematiksel mantığı görünür kılması bakımından önemlidir. Özellikle eğitim amaçlı çalışmalarda, hazır bir fonksiyonun iç işleyişini kavramaya yardımcı olmaktadır.

3.4. NumPy ile karşılaştırma

Aynı matris için NumPy yöntemi ile elde edilen özdeğerler:

$$[5, 3, 7]$$

manuel yöntem ile elde edilen özdeğerler ise:

[7, 5, 3]

şeklindedir.

Bu iki sonuç aynı özdeğer kümesini vermektedir. Aradaki fark yalnızca sıralamadan kaynaklanmaktadır. NumPy dokümantasyonunda özdeğerlerin belirli bir sırada döndürülmesinin garanti edilmediği açıkça ifade edildiğinden, bu farklılık doğal ve beklenen bir durumdur. Dolayısıyla iki yöntem de aynı matris için doğru sonuç üretmiştir.

3.5. Performans değerlendirmesi

Karşılaştırma sonucunda NumPy tabanlı hesaplamanın manuel yöntemle göre daha kısa sürede sonuç verdiği görülmüştür. Küçük boyutlu matrislerde bu fark sınırlı düzeyde olsa da manuel yöntemin determinant açılımı ve polinom işlemleri nedeniyle daha yüksek hesaplama maliyetine sahip olduğu açıktır. Matris boyutu arttıkça bu farkın daha da büyümesi beklenmektedir.

Manuel yöntem, karakteristik polinom temelli teorik yaklaşımı açık biçimde göstermesi bakımından değerlidir. Buna karşın uygulama verimliliği açısından NumPy'nin sayısal lineer cebir tabanlı yaklaşımı daha güçlüdür. Bu nedenle eğitimsel amaçlarla manuel yaklaşımın, gerçek uygulamalarda ise NumPy gibi optimize edilmiş kütüphanelerin tercih edilmesi daha uygundur.

3.6. Sonuçların yorumu

Elde edilen bulgular, manuel karakteristik polinom yaklaşımının ve NumPy'nin hazır özdeğer hesaplama yönteminin aynı matematiksel sonuca ulaştığını göstermektedir. Bununla birlikte NumPy, arka planda gelişmiş sayısal algoritmalar kullandığından daha verimli ve ölçeklenebilir bir çözüm sunmaktadır.

Manuel yaklaşım ise özellikle özdeğer kavramının nasıl ortaya çıktığını ve determinant temelli çözüm mantığını anlamak açısından öğreticidir. Bu bağlamda iki yöntem birbirinin alternatifi olmaktan çok, farklı amaçlara hizmet eden tamamlayıcı yaklaşımlar olarak değerlendirilebilir.

Sonuç

Bu çalışmada matris manipülasyonu, özdeğerler ve özvektörlerin makine öğrenmesindeki yeri incelenmiş; NumPy'nin `linalg.eig` fonksiyonu dokümantasyon ve kaynak kod düzeyinde değerlendirilmiş; son olarak manuel özdeğer hesabı ile hazır kütüphane yaklaşımı karşılaştırılmıştır.

İnceleme sonucunda, matrislerin makine öğrenmesi problemlerinin temel temsil biçimi olduğu; özdeğer ve özvektörlerin ise özellikle boyut indirgeme, veri analizi ve doğrusal dönüşümlerin yorumlanmasında önemli rol oynadığı görülmüştür. NumPy'nin `eig` fonksiyonu, bu hesabı kullanıcı açısından kolaylaştıran ve güçlü sayısal altyapıya dayanan bir araçtır.

Manuel karakteristik polinom yaklaşımı, teorik anlayış kazandırma bakımından oldukça yararlıdır. Ancak performans, kararlılık ve ölçeklenebilirlik açısından hazır sayısal kütüphaneler daha uygun bir çözüm sunmaktadır. Bu nedenle öğretim ve kavramsal

kavrama amacıyla manuel yöntemlerin, uygulama ve mühendislik bağlamında ise NumPy gibi optimize edilmiş kütüphanelerin tercih edilmesi yerinde olacaktır.

Kaynakça

Brownlee, J. *Introduction to Matrices and Matrix Arithmetic for Machine Learning*. Machine Learning Mastery.

Brownlee, J. *Gentle Introduction to Eigenvalues and Eigenvectors for Machine Learning*. Machine Learning Mastery.

NumPy Documentation. *numpy.linalg.eig*.

NumPy Source Code. *numpy/linalg/_linalg.py*.

LucasBN. *Eigenvalues-and-Eigenvectors* GitHub Repository.

<https://www.geeksforgeeks.org/engineering-mathematics/eigen-values/>